

Lesson 07: Blocking Ultrasonic Distance

An HC-SR04 ultrasonic sensor sends a short burst of sound that is too high for people to hear. Its **Echo** signal stays HIGH while that sound travels to a target and back. This lesson measures that round-trip delay, calculates the one-way distance, reports both values, and then deliberately waits 500 milliseconds before measuring again.

Recommended Prior Learning

Complete [lesson 06: Joystick Motor and Servo Control](#) first when following the main curriculum route. You should already recognize Arduino input and output pins, milliseconds and microseconds, Serial Monitor reports, and a program that waits between reports.

Lesson 07 is a new standalone circuit. Disconnect and set aside lesson 06's joystick, motor, servo, MOSFET circuit, and external actuator supply. This lesson uses only the USB-powered Uno and HC-SR04.

What You Need

- Arduino Uno and USB data cable;
- four-pin, 5 V HC-SR04 ultrasonic sensor labeled **VCC**, **Trig**, **Echo**, and **GND**;
- four suitable jumper wires;
- breadboard or stable support if needed; and
- a flat, firm target such as a book or piece of stiff cardboard.

Core Mission: Compare a Near and Far Target

Goal: Hold the same target at two distances and compare the reported Echo delay and calculated distance.

Prepare and stay safe: Unplug USB before connecting or moving any wire. Follow the labels printed on your sensor; do not guess pin order from a picture or from which way the board faces. This lesson has no external power supply and no moving actuator. Keep the Uno and sensor still, and move only the target without bumping wires. Do not place the sensor against a person or point it directly into someone's ear. If any part becomes hot, smells unusual, smokes, sparks, or behaves unexpectedly, unplug USB immediately and tell the teacher. Do not reconnect until the setup has been checked.

1. With **USB unplugged**, make these four connections:

HC-SR04 label	Arduino Uno	Purpose
VCC	5 V	Powers the 5 V sensor
Trig	D4	Starts one measurement
Echo	D2 / INT0	Returns a pulse whose width represents round-trip time
GND	GND	Completes the circuit and provides the shared reference

2. **Teacher checkpoint, still unpowered:** Trace every connection by its printed label. Confirm VCC reaches Arduino 5 V, GND reaches Arduino GND, Trig reaches D4, and Echo reaches D2/INT0. Check for

loose strands, shifted header connections, or a direct 5 V-to-GND short. Stabilize the sensor so its two round transducers face an open area. Restore USB power only after approval. Lesson 07 reads the pulse directly; lessons 08 and 09 will use D2's INT0 function.

3. Open `platformio-uno-lessons/07-ultrasonic-blocking-distance` as the PlatformIO project in VS Code. Under **Project Tasks > uno > General**, choose **Build** and wait for **SUCCESS**. Close any open Monitor, connect USB, choose **Upload**, and wait for **SUCCESS**.
4. Open **Monitor** at **9,600 baud**. Hold the flat target approximately 10 cm in front of the sensor without touching the sensor or wires. Keep the target face roughly parallel to the sensor face. Read several lines such as:

```
Echo delay: 0.580 ms | Distance: 10.0 cm
```

5. Record one reasonably steady near reading. Move the **same target** to approximately 50 cm, keep it facing the sensor, and record one reasonably steady far reading. Ordinary readings may vary slightly.

Target position	Echo delay (ms)	Calculated distance (cm)
Near		
Far		

6. Compare the two rows. The farther target should usually produce both a longer Echo delay and a larger calculated distance. If the monitor instead reports **Echo: invalid/no echo within 25.000 ms**, use the troubleshooting section without rewiring while powered.

Notice: Did the farther target have the longer round-trip delay? About how many times larger was its distance than the near reading? Record what you actually observed, even if the readings moved around.

Stop safely: Close Monitor and unplug USB. Change or remove wires only after USB power is disconnected.

Name the Idea: Measure Travel Time

The sensor does not directly send the Arduino a number of centimeters. The Arduino sends a short pulse on D4, and the sensor responds with a HIGH pulse on D2. The program measures how many microseconds D2 stays HIGH.

That time covers a trip to the target **and back**:

```
sensor  ——— sound travels out ———>  target
sensor  <—— echo travels back  ——  target
```

Using the common HC-SR04 approximation, 58 microseconds of round-trip Echo time represents about 1 centimeter of one-way distance:

```
distance in centimeters = Echo duration in microseconds / 58
```

For example, `580 us / 58 = 10 cm`. The displayed delay is the same 580 microseconds written as `0.580 ms`.

Predict and Try One Bounded Change

Predict what will happen to the Echo delay if you move the target from about 20 cm to about 40 cm. With the approved wiring unchanged, reconnect USB, open Monitor, and try only those two target positions. Do not expect perfect doubling: target angle, size, surface, nearby objects, air conditions, and inexpensive sensor variation can affect the result.

Stop Monitor and unplug USB when finished.

Why This Lesson Is Blocking

The focus sequence in `src/main.cpp` is:

```
send Trigger pulse
wait for the Echo pulse
print the result
wait 500 ms
repeat
```

`pulseIn(...)` waits while the Echo pulse begins and ends. `delay(500)` then makes the program wait another half second. A valid reading can therefore appear a little more than 500 ms after the previous reading. This blocking behavior is intentional; a later lesson will compare it with interrupt-driven Echo capture.

The Echo wait has an explicit **25,000 microsecond (25 ms) timeout**. The HC-SR04's advertised 4 m maximum corresponds to about 23.2 ms of round-trip Echo time, so 25 ms provides a small margin. The timeout prevents the program from using the much longer default wait when no complete Echo arrives.

Measurement Limits

- The usual advertised range is approximately 2–400 cm; results outside it are not promised.
- A broad, hard, flat target facing the sensor usually reflects sound better than cloth, a narrow object, or an angled surface.
- Nearby walls and objects can create other echoes.
- This is a learning measurement, not calibrated precision equipment and not a safety or collision-avoidance system.
- **Echo: invalid/no echo** means no complete pulse was measured before the timeout. It does **not** mean the target is zero centimeters away.

The [teacher guide](#) gives the timing calculation, preparation checklist, expected observations, and implementation boundary.

Build and Host-Test Commands

From this lesson directory, build the Uno firmware with:

```
pio run
```

The distance math can also be checked on a computer without an Uno or sensor:

```
g++ -std=c++11 -Wall -Wextra -Werror -Iinclude  
test/test_ultrasonic_math.cpp -o /tmp/lesson07-ultrasonic-check  
/tmp/lesson07-ultrasonic-check
```

Expected host-test result:

```
Ultrasonic checks passed: validity, time, 10 cm, and 4,000 mm range.
```

Troubleshooting and Stop Conditions

- **Nothing appears in Monitor:** Confirm Build and Upload both ended with **SUCCESS**, select the **uno** environment's Monitor task, and verify 9,600 baud. Close another program that may be using the serial port, then reopen Monitor.
- **Every line says invalid/no echo:** Keep wires untouched while powered. First use a flat hard target 10–30 cm in front of the sensor. If it still fails, unplug USB and ask the teacher to trace VCC → 5 V, GND → GND, Trig → D4, and Echo → D2/INT0 by the sensor's printed labels.
- **The number jumps:** Hold the sensor and target still, face the target toward both transducers, and move other objects away from the sound path. Small changes are normal.
- **The measured distance is much too short:** Nearby objects may be reflecting the sound first. Clear the space in front and to the sides of the sensor.
- **The measured distance is much too long or intermittent:** Move the target closer, use a larger and harder flat surface, and reduce its angle. Soft or angled targets may not return a strong Echo.
- **The near and far values seem reversed:** Confirm you recorded fresh lines after moving the target. Keep the target still for several reports at each position.
- **A part becomes hot, smells unusual, sparks, or smokes:** Unplug USB immediately. Do not touch a hot part or reconnect power; tell the teacher and inspect the 5 V/GND connections.

Next, continue to [lesson 08: Interrupt Ultrasonic and Joystick Control](#) to capture Echo edges while joystick motor/servo control continues.